

MOVIE RECOMMENDATION SYSTEM USING DEEP LEARNING COLLABORATIVE FILTERING TO IMPROVE RECOMMENDATION PERFORMANCE

Tran Thanh Phong (K18 HCM)
FPT University
phongttse184744@fpt.edu.vn

February 2025

Abstract

Movie recommendation systems play a crucial role in enhancing user experience by suggesting relevant movies based on user preferences. Traditional collaborative filtering approaches suffer from data sparsity and cold start problems, which limit their effectiveness. Similarly, content-based filtering, while effective in recommending movies based on individual user preferences, faces challenges with cold start issues for new items and a lack of recommendation diversity.

To address these challenges, I propose to use Sparse Autoencoder combined with Neural Collaborative Filtering to improve the performance of rating and ranking prediction. First, I use Sparse Autoencoder trained on user-item interactions matrix to generate sparse latent representation of users, with sparsity constraint to learn important features and reduce redundancy. Then, it is used to initialize user embedding for NeuralCF model. NCF is trained with dynamic total loss, balancing between rating loss (MSE) and ranking loss (hinge loss), integrating hard negative sampling to improve ranking quality.

Advantages: Learning efficient feature representations from sparse data, Enhanced ability to model nonlinear relationships, Minimize cold-start and sparsity issues, High extensibility and generality

Introduction

Movie recommendation systems are an important application of artificial intelligence (AI) and machine learning (ML), helping to personalize content for users. Platforms such as Netflix, Disney+, Hulu, YouTube all use this technology to retain users and optimize the movie viewing experience.

In the digital age, digital content is increasingly expanding with millions of movies, TV shows and videos on online platforms. Finding suitable content is becoming more difficult for users. Therefore, movie recommendation systems are developed to personalize the movie viewing experience, helping users discover new content that they may be interested in.

Big and Sparse Data: The MovieLens 10M dataset contains 10 million ratings from thousands of users across thousands of movies. Each user only rates a handful of movies, making the data sparse. -Cold-start problem: When there is a new user or new movie, the system does not have enough data to make accurate recommendations. -Performance on constrained hardware: Processing large datasets on resource-limited CPUs or GPUs requires model optimization. -High personalization: Not everyone has the same preferences, so the model needs to learn the hidden features of each user.

To solve this problem, I propose a hybrid model that combines: - Sparse autoencoders help learn efficient feature representations from sparse data, mitigate cold start and sparsity issues - Neural Collaborative Filtering (NCF) enhances the ability to model nonlinear relationships, high extensibility and generality

The paper aims to overcome the limitations of traditional collaborative filtering methods (such as matrix factorization) when they cannot model non-linear relationships between users and items. Integrating nonlinear capabilities using deep neural network architectures, hard negative sampling strategy allows for efficient selection of “difficult” samples, combine two tasks for predicting ratings and rankings using a loss function, NeuralCF model using deep learning architectures combining Embeddings and Multilayer Perceptrons to learn user-product interactions

Related Works

Part 1: Projects that solve the same problem as me

Group 1: Collaborative Filtering - CF method

- Typical projects:
 - Sarwar et al. (2001) - “Item-based collaborative filtering recommendation algorithms” (ACM)
 - Koren, Bell Volinsky (2009) – “Matrix Factorization Techniques for Recommender Systems” (Computer).
- Advantages:
 - Based on collective experience: Mining data from many users, it is possible to capture “collective intelligence” that helps accurately predict individual preferences.

- Modern modeling capabilities: Model-based methods (such as matrix decomposition, SVD, SVD++) are often accurate and scalable given sufficient data.
- Disadvantages:
 - Cold-start: When a new user or item (without ratings) comes in, the system struggles due to lack of interaction data.
 - Sparsity: With a very large number of movies and each user only rating a small portion, the interaction matrix is often very sparse, making it difficult to calculate similarity.
 - Limited interpretability: Especially with model-based methods, the “latent factors” are not always semantically understandable.

Group 2: Content-Based Filtering – CBF method

- Typical projects:
 - Pazzani, M. J. Billsus, D. (2007). “Content-Based Recommendation Systems.
 - Lops, P., Gemmis, M., Semeraro, G. (2011). ‘Content-Based Recommender Systems: State-of-the-art and trending. Ricci, F., Rokach, L., Shapira, B., Kantor, P. B. (Eds.). (2011). ‘Recommender Systems Handbook’.
- Advantages:
 - No dependence on other users’ data: The system only needs meta-data information of items (movies) such as genre, director, actors, description, etc. to help solve the cold-start problem for items (item cold-start).
 - Personalized recommendations: Based on the user’s viewing history or movies that they have liked, the system can recommend movies with similar characteristics, suitable for each user’s personal preferences.
 - Easily scalable with quality metadata: With rich descriptive data, the system can learn important movie features to make accurate predictions.
- Disadvantages:
 - Overspecialization: The system tends to only recommend movies that are similar to the movies the user has already watched, resulting in a lack of diversity and the ability to explore new genres.

- Depends on the quality of the description data: If the metadata information is incomplete or corrupted, the quality of the recommendation will decrease. Labeling and updating information for each movie requires a lot of effort and resources.
- Failing to capitalize on user behavior: Because it relies solely on movie attributes, this approach ignores signals from user behavior and ratings (which CF typically exploits), thus missing out on some valuable insights from “collective intelligence.”

Group 3: Hybrid Methods (combines the advantages of Collaborative Filtering (CF) and Content-Based Filtering (CBF))

- Typical projects:
 - Burke, R. (2002). “Hybrid Recommender Systems: Survey and Experiments.”
 - Adomavicius, G., Tuzhilin, A. (2005). “Toward the Next Generation of Recommender Systems: A Survey of the State-of-the-Art and Possible Extensions.” *IEEE Transactions on Knowledge and Data Engineering*, 17(6), 734–749.
 - Bobadilla, F., Ortega, F., Hernando, A., Gutiérrez, A. (2013). “Recommender Systems Survey.” *Knowledge-Based Systems*, 46, 109–132.
- Advantages:
 - Combining the advantages of CF and CBF: The hybrid approach integrates the ability to mine information from user review history (CF) with the use of movie metadata information (CBF). This helps to compensate for the limitations of each individual method, creating more accurate and personalized recommendations.
 - Mitigating cold-start issues: When new users or new movies do not have enough interaction data, descriptive information from the movie content side can be used to generate initial recommendations, mitigating the limitations of CF.
 - Increasing the diversity of recommendations: Combining data sources (CF, CBF, even contextual data or expert reviews) helps the system not only recommend “well-worn” movies but also encourages the exploration of other movie genres, thereby improving the user experience.
 - Highly customizable: Hybrid systems can be tuned to balance the weights between different components, suitable for each specific application context, helping to achieve more optimal performance in different scenarios.

- Disadvantages:
 - High complexity: Integrating multiple models (CF, CBF, and possibly adding contextual data, expert judgment...) leads to complex system architectures, requiring large computational costs and careful design.
 - Optimization and parameter tuning costs: Balanced weighting between the components of a hybrid system is a non-trivial task. This requires testing, optimization, and can be resource intensive (in terms of time and computation) to achieve the optimal configuration.
 - High-quality input data requirements: The effectiveness of the hybrid system depends on the quality of both user data (CF) and movie description information (CBF). If either of them is corrupted or incomplete, the recommendation quality can be severely affected.

Group 4: Context-Aware / Emotion-Based Approaches

- Typical projects:
 - Yerkin et al. (2024) – “Multi-channel Emotion Analysis for Consensus Reaching in Group Movie Recommendation Systems” (arXiv:2404.13778).
 - Varma Petluri (2021) – “Movie Recommender System using critic consensus” (arXiv:2112.11854).
- Advantages:
 - Deeper personalization: These systems not only take into account review history but also use contextual information (such as time, location, user emotional state) or opinions from experts, making suggestions more relevant to the specific situation (especially in group recommendations).
 - Group recommendation support: Suitable for group movie watching situations, when it is necessary to satisfy the emotional preferences of many users at the same time
- Disadvantages:
 - Complex data requirements: Data collection from multiple channels (text, audio, images) and integrated processing are required, which is technically difficult and costly.
 - Complex assessment and synthesis: Determining the “level of consensus” in a group or the “dominant sentiment” requires complex models such as fuzzy inference systems to combine sentiment scores from multiple sources.

Part 2: Projects using the same method as mine

Group 1: Pure Deep Collaborative Filtering Model

- Typical projects:
 - Neural Collaborative Filtering (NCF) – He et al., WWW 2017
- Advantages:
 - High non-linear modeling capabilities: Use MLP to learn complex interactions between users and items.
 - Simple and direct structure: Helps optimize rating or ranking prediction tasks.
- Disadvantages:
 - Requires large data: To avoid overfitting and achieve good performance, the model needs a rich amount of interaction data.
 - Limited interpretability: Nonlinear relationships learned by neural networks are difficult to interpret compared to linear models.
 - High computational cost: requirements (gpu,cpu,memory) increase

Group 2: Wide and Deep Recommender Systems

- Typical projects:
 - Wide Deep Learning for Recommender Systems – Cheng et al., WSDM 2016
- Advantages:
 - Combining the advantages of linear models and deep learning: The wide part helps to remember common relationships, while the deep part explores complex nonlinear interactions.
 - Cold-start handling: When interaction data is limited, the wide part can help the model make more stable predictions.
- Disadvantages:
 - Complex architecture: Combining two components requires careful design and hyperparameter optimization.
 - Increased computational cost: Due to the complexity of the model, training can be time-consuming and resource-intensive.

Group 3: Factorization Machines Autoencoder-Based Approaches

- Typical projects:
 - DeepFM: A Factorization-Machine based Neural Network for CTR Prediction – Guo et al., IJCAI 2017
 - Neural Factorization Machines for Sparse Predictive Analytics – He et al., IJCAI 2017
 - Variational Autoencoders for Collaborative Filtering – Liang et al., WWW 2018
- Advantages:
 - Efficient Sparse Data Handling: Using techniques like autoencoders or VAEs helps learn quality hidden representations from sparse interaction matrices
 - Modeling nonlinear interactions: Combining traditional models (like FM) with deep neural networks helps exploit complex relationships between features.
 - High representation capability: Autoencoder allows learning deep latent features and can integrate other auxiliary information.

Methodology

Movie recommendation system: Using deep collaborative filtering with sparse autoencoder to solve persistent challenges The movie recommendation system leverages a deep collaborative filtering with sparse autoencoder model integrated with deep learning techniques to address the challenges outlined in the Introduction. This model is primarily developed using the PyTorch library, which offers robust capabilities for handling large-scale datasets. Although PyTorch excels at managing extensive data, its integration of deep learning is particularly advantageous for training models with Movielens-10m data. This is especially significant given that PyTorch is developed by Meta, the corporation behind Facebook, the world's largest social media platform. Meta processes vast amounts of data daily, and its recommendation systems are highly optimized to accommodate this scale, ensuring the best possible user experience on its social media platforms.

In constructing the deep collaborative filtering model, several optimization algorithms are employed to enhance performance, including:

Input

- $\text{User}(\text{user}_i d, \text{user}_i \text{informationage}, \text{sex})$.
- $\text{Ratings}(\text{user}_i d, \text{movie}_i d, \text{rating}, \text{timestamp})$.
- $\text{Movie}(\text{movie}_i d, \text{title}, \text{genres})$

Output

Constraints of the problem

Limited computing resources
Big Data(Movielens 10M)
Accuracy and variety
Fast prediction time
Availability for new users(Cold Start problem)

Algorithm

In Autoencoder

SparsityLoss: The sparsity loss enforces sparse activations in a neural network by minimizing the KL-divergence between the average activation and a predefined sparsity target p .

$$\bar{a} = \frac{1}{m} \sum_{i=1}^m a_i$$

- $\bar{a}$ represents the **average activation** of a hidden neuron over m input samples.
- This step calculates how frequently a neuron is activated across all training samples.

(1) **Compute average activation:** $\bar{a}$ represents the average activation of a hidden neuron over m samples.

$$\bar{a} = \text{clamp}(\bar{a}, 10^{-6}, 1 - 10^{-6})$$

- This step ensures numerical stability by preventing $\bar{a}$ from becoming exactly 0 or 1.
- Avoids undefined logarithms in the next step, as $\log(0)$ is not defined.

(2) Clamping activation values: This step prevents numerical instability by ensuring that $\bar{a}$ does not reach 0 or 1, avoiding undefined logarithms.

$$D_{KL}(p \parallel \bar{a}) = p \log \frac{p}{\bar{a}} + (1 - p) \log \frac{1 - p}{1 - \bar{a}}$$

- Measures the difference between the average activation $\bar{a}$ and the predefined sparsity target p .
- If $\bar{a}$ closely matches p , the KL-divergence value is small, meaning activations follow the desired sparsity.
- Encourages neurons to activate only when necessary, preventing overfitting and redundancy.

(3) KL-Divergence Calculation: Measures how different the average activation $\bar{a}$ is from the sparsity target p . A lower value indicates that the activations match the desired sparsity.

$$L_{\text{sparsity}} = \lambda \sum_j D_{KL}(p \parallel \bar{a}_j)$$

(4) Total sparsity loss: Summing the KL-divergence across all hidden neurons and multiplying by a weight λ to control the regularization effect.

- The total sparsity loss is computed by summing the KL-divergence across all hidden neurons.
- λ is a hyperparameter that controls the strength of the sparsity regularization.
- A higher λ forces the model to maintain sparsity more strictly.
- This loss is added to the overall loss function to balance between reconstruction accuracy and sparsity in the hidden layer.

ReconstructionLoss:

$$L_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

$$L_{\text{sparsity}} = \lambda \sum_j D_{KL}(p \parallel \bar{a}_j)$$

$$L_{\text{total}} = L_{\text{MSE}} + L_{\text{sparsity}}$$

- L_{MSE} measures the reconstruction error of the autoencoder.
- x_i represents the **original input data** (ground truth).

- $\hat{x}_i$ represents the **reconstructed output** produced by the autoencoder.
- The squared error $(x_i - \hat{x}_i)^2$ penalizes larger errors more heavily than smaller ones.
- The sum is averaged over all n samples to normalize the loss.
- The objective is to minimize L_{MSE} to make $\hat{x}_i$ as close as possible to x_i .

NeuralCF

1. User and Item Embeddings Let:

$$E_u \in \mathbb{R}^{N_u \times d} \quad \text{and} \quad E_i \in \mathbb{R}^{N_i \times d}$$

be the embedding matrices for users and items, where:

- N_u is the number of users.
- N_i is the number of items.
- d is the embedding dimension.

For a given user u and item i , the embeddings are:

$$e_u = E_u[u] \quad \text{and} \quad e_i = E_i[i]$$

Explanation:

- $e_u \in \mathbb{R}^d$ is the latent representation (embedding) of user u .
- $e_i \in \mathbb{R}^d$ is the latent representation (embedding) of item i .
- These embeddings capture the underlying features of users and items.

2. Context Vector Formation The context vector is obtained by concatenating the user and item embeddings:

$$c = \begin{bmatrix} e_u \\ e_i \end{bmatrix} \in \mathbb{R}^{2d}$$

Explanation:

- The notation $\begin{bmatrix} e_u \\ e_i \end{bmatrix}$ denotes the concatenation of e_u and e_i .
- The resulting vector c combines user and item features and serves as input to the subsequent MLP layers.

3. Rating Prediction Network (MLP for Rating) The rating prediction is computed via a multi-layer perceptron (MLP) that processes the context vector c :

$$\hat{r} = f_{\text{rating}}(c) = W_r^{(L)} \sigma \left(W_r^{(L-1)} \sigma \left(\dots \sigma \left(W_r^{(1)} c + b_r^{(1)} \right) \dots \right) + b_r^{(L-1)} \right) + b_r^{(L)}$$

Explanation:

- $\hat{r}$ is the predicted rating.
- $W_r^{(l)}$ and $b_r^{(l)}$ are the weight matrix and bias vector of the l -th layer in the rating MLP.
- $\sigma(\cdot)$ denotes an activation function (e.g., LeakyReLU).
- The MLP transforms the context vector c through multiple non-linear layers to output a rating prediction.

4. Ranking Prediction Network (MLP for Ranking) Similarly, the ranking prediction is computed using a separate MLP:

$$\hat{z} = f_{\text{ranking}}(c) = W_z^{(L)} \sigma \left(W_z^{(L-1)} \sigma \left(\dots \sigma \left(W_z^{(1)} c + b_z^{(1)} \right) \dots \right) + b_z^{(L-1)} \right) + b_z^{(L)}$$

Explanation:

- $\hat{z}$ is the ranking score for the given user-item pair.
- $W_z^{(l)}$ and $b_z^{(l)}$ are the weights and biases of the l -th layer in the ranking MLP.
- The ranking MLP processes the same context vector c to output a score that can be used for ranking items.

Combinedloss for training model

1. Rating Loss (Mean Squared Error) The rating loss is defined as:

$$L_{\text{rating}} = \frac{1}{n} \sum_{i=1}^n (\hat{r}_i - r_i)^2$$

Explanation:

- $\hat{r}_i$ is the predicted rating for the i -th sample.
- r_i is the ground truth (actual rating) for the i -th sample.
- n is the total number of samples.
- This loss measures the squared error between the predicted ratings and the actual ratings, penalizing larger discrepancies more heavily.

2. Ranking Loss (Margin Ranking Loss) The ranking loss is computed via a margin ranking approach:

$$L_{\text{ranking}} = \frac{1}{n} \sum_{i=1}^n \max\{0, m - (\hat{z}_i^+ - \hat{z}_i^-)\}$$

where m is the margin (here $m = 1.0$). **Explanation:**

- $\hat{z}_i^+$ represents the ranking score for the positive (relevant) item in the i -th sample.
 - $\hat{z}_i^-$ represents the ranking score for the negative (irrelevant) item in the i -th sample.
 - The margin m enforces that the positive item should have a score at least m higher than the negative item.
 - The $\max\{0, \cdot\}$ operation ensures that only samples violating the margin (i.e., where $\hat{z}_i^+ - \hat{z}_i^- < m$) contribute to the loss.
3. **Regularization Loss** The regularization loss is computed as the sum of the L2 norms of all model parameters:

$$L_{\text{reg}} = \sum_{p \in \theta} \|p\|_2$$

Explanation:

- θ denotes the set of all parameters in the model.
 - $\|p\|_2$ is the L2 norm (Euclidean norm) of a parameter p .
 - This term penalizes large weights, which helps prevent overfitting by encouraging simpler models.
4. **Total Combined Loss** The overall loss function is a weighted sum of the rating loss, ranking loss, and regularization loss:

$$L_{\text{total}} = L_{\text{rating}} + \alpha L_{\text{ranking}} + \lambda L_{\text{reg}}$$

where:

- α (ranking_weight) scales the ranking loss.
- λ (lambda_reg) scales the regularization loss.

Explanation:

- The total loss function balances three objectives:
 - Accurately predicting ratings (via L_{rating}).
 - Ensuring a proper ranking of positive over negative items (via L_{ranking}).
 - Maintaining model generalizability and preventing overfitting (via L_{reg}).
- By minimizing L_{total} , the model is simultaneously optimized for accurate reconstruction (ratings), effective ranking, and controlled model complexity.

Evaluation metrics

0.0.1 Precision@K

Precision measures the proportion of recommended items that are relevant:

$$\text{Precision@K} = \frac{\text{Number of relevant items in top-K recommendations}}{K}$$

where relevant items are defined as movies with user ratings greater than 3 (on a scale of 1-5).5).5).5).

0.0.2 Recall@K

Recall measures the proportion of relevant items retrieved in the top-K recommendations:

$$\text{Recall@K} = \frac{\text{Number of relevant items in top-K recommendations}}{\text{Total number of relevant items}}$$

This reflects the coverage of all relevant movies a user likes in the recommendations.

0.0.3 Mean Reciprocal Rank (MRR@K)

MRR evaluates the ranking quality of the first relevant item:

$$\text{MRR@K} = \frac{1}{Q} \sum_{q=1}^Q \frac{1}{\text{rank}_q}$$

where Q is the number of users, and rank_q is the position of the first relevant item (rating ≥ 3) in the top-K list for user q . If no relevant item exists, $\text{rank}_q = \infty$, contributing 0 to MRR.

0.0.4 Normalized Discounted Cumulative Gain (NDCG@K)

NDCG measures the ranking quality, accounting for the position of relevant items with logarithmic decay:

$$\text{DCG@K} = \sum_{i=1}^K \frac{\text{rel}_i}{\log_2(i+1)}$$

$$\text{IDCG@K} = \sum_{i=1}^K \frac{\text{rel}_i^{\text{ideal}}}{\log_2(i+1)}$$

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

where rel_i is the relevance score of the item at position i (e.g., 1 for ratings ≥ 3 , 0 otherwise, or scaled by rating), and $\text{rel}_i^{\text{ideal}}$ is the relevance in the ideal ranking.

Schedule code

// Purpose: Build and test a recommendation system for movies using NCF and MovieLens 10M data

// Input: MovieLens 10M dataset (ratings, users, movies), top recommendations ($K = 10$), // embedding size = 256, batch size = 512, training cycles = 15, learning rate = 0.001 // Output: Trained model, movie recommendations, accuracy scores (Precision, Recall, MRR, NDCG)

// Step 1: Prepare the Data Load MovieLens 10M data from files or online
Read ratings, movies, and tags into tables Clean data: remove unnecessary details (like timestamps), number users and movies Split data into training (80% Create data batches for processing with size 512

1 Experiment